Schroer-Ott targeting for the lifted standard map

This Live Script rebuilds the corrected Schroer-Ott workflow from the computational pieces rather than calling `main_schroer_ott_targeting`.

The state space used here is the lifted cylinder S^1 x R: x is periodic modulo one, while y is never wrapped. Controls are instantaneous vertical y-kicks with u <= 0.003. The operational targeting method uses small proxy balls around direct-hyperbolic unstable periodic orbits (UPOs), then chooses switching iterates by minimizing the remaining resolved transfer time to the next proxy or final target.

Stable and unstable manifolds are diagnostic overlays. They are not used as a complete explicit polygonal pass network.

```
clear; close all; clc;
projectFolder = fileparts(mfilename('fullpath'));
restoredefaultpath;
rehash toolboxcache;
addpath(projectFolder);
fprintf('Project folder: %s\n', projectFolder);
```

```
Project folder: C:\Users\npcoz\AppData\Local\Temp\Editor_ywhkb
```

```
fprintf('MATLAB version: %s\n', version);
```

```
MATLAB version: 26.1.0.3276743 (R2026a) Update 3
```

Authoritative implementation and file inventory

Only this folder is placed on the MATLAB path. The nested backup/copy folder is deliberately not added to the path. The inventory table below is generated from the files that MATLAB can see in this authoritative folder.

```
authoritativeFolder = projectFolder;
authoritativeFunctions = {
    'schroer_ott_default_config'
    'so_standard_map_lifted'
    'so_standard_map_inverse_lifted'
    'so_jacobian'
    'so_jacobian_product'
    'so_enumerate_periodic_orbits'
    'so_construct_first_light_route'
    'so_resolve_connection'
    'so_multistage_targeting'
    'so_replay_stage'
    'so_plot_results'
    };
calledFiles = strings(numel(authoritativeFunctions), 1);
for k = 1:numel(authoritativeFunctions)
    calledFiles(k) = string(which(authoritativeFunctions{k}));
end
calledFileTable = table(string(authoritativeFunctions), calledFiles, ...
    'VariableNames', {'functionName','resolvedPath'})
```

calledFileTable = 11×2 table

	functionName	resolvedPath
1	"schroer_ott_default_config"	"C:\Users\npcoz\OneDrive - Imperial College London\Project\Codes\schroer_ott_targeting\schroer_ott_default_config.m"
2	"so_standard_map_lifted"	"C:\Users\npcoz\OneDrive - Imperial College London\Project\Codes\schroer_ott_targeting\so_standard_map_lifted.m"
3	"so_standard_map_inverse_lifted"	"C:\Users\npcoz\OneDrive - Imperial College London\Project\Codes\schroer_ott_targeting\so_standard_map_inverse_lifted.m"
4	"so_jacobian"	"C:\Users\npcoz\OneDrive - Imperial College London\Project\Codes\schroer_ott_targeting\so_jacobian.m"
5	"so_jacobian_product"	"C:\Users\npcoz\OneDrive - Imperial College London\Project\Codes\schroer_ott_targeting\so_jacobian_product.m"
6	"so_enumerate_periodic_orbits"	"C:\Users\npcoz\OneDrive - Imperial College London\Project\Codes\schroer_ott_targeting\so_enumerate_periodic_orbits.m"
7	"so_construct_first_light_route"	"C:\Users\npcoz\OneDrive - Imperial College London\Project\Codes\schroer_ott_targeting\so_construct_first_light_route.m"
8	"so_resolve_connection"	"C:\Users\npcoz\OneDrive - Imperial College London\Project\Codes\schroer_ott_targeting\so_resolve_connection.m"
9	"so_multistage_targeting"	"C:\Users\npcoz\OneDrive - Imperial College London\Project\Codes\schroer_ott_targeting\so_multistage_targeting.m"
10	"so_replay_stage"	"C:\Users\npcoz\OneDrive - Imperial College London\Project\Codes\schroer_ott_targeting\so_replay_stage.m"
11	"so_plot_results"	"C:\Users\npcoz\OneDrive - Imperial College London\Project\Codes\schroer_ott_targeting\so_plot_results.m"

```
inventory = local_file_inventory(projectFolder)
```

inventory = 2×4 table

	name	folder	bytes	lastModified
1	"LiveEditorEvaluationHelperEeditor9BDAE587motw.m"	"C:\Users\npcoz\AppData\Local\Temp\Editor_ywhkb"	22376	17-Aug-2026 00:24:32
2	"LiveEditorEvaluationHelperEeditorEDA19B58motw.m"	"C:\Users\npcoz\AppData\Local\Temp\Editor_ywhkb"	11731	17-Aug-2026 03:33:14

Mathematical and numerical configuration

The corrected deterministic first-light case is not the paper Figure-3 geometry. It is a nondegenerate development case at k=1.25. Its source centre is outside every retained proxy ball and remains outside those balls under every admissible zero-time vertical control.

The route bracket has a declared margin and open boundaries. This avoids accepting or rejecting a resonance merely because a rectangle centre is equal to a nominal rotation number to round-off precision.

```
cfg = schroer_ott_default_config();
cfg.saveFigures = true;
if ~exist(cfg.outputDirectory, 'dir'); mkdir(cfg.outputDirectory); end
if ~exist(cfg.figureDirectory, 'dir'); mkdir(cfg.figureDirectory); end
configurationTable = local_configuration_table(cfg)
```

configurationTable = 16×2 table

	quantity	value
1	"k"	"1.25"
2	"controlAmplitude"	"0.003"
3	"sourceX"	"[0.859409, 0.909409]"
4	"sourceY"	"[0.486716, 0.526716]"
5	"targetX"	"[0.210000, 0.250000]"
6	"targetY"	"[0.720000, 0.760000]"
7	"maxForwardIterations"	"8"
8	"maxBackwardIterations"	"6"
9	"maxTotalTransferTime"	"14"
10	"proxyTargetRadius"	"0.03"
11	"routeBracketMargin"	"1e-10"
12	"routeLowerBoundary"	"open"
13	"routeUpperBoundary"	"open"
14	"intersectionBackend"	"indexed"
15	"equationOneTargetSizeConvention"	"diameter"
16	"equationOneLConvention"	"declared order-one phase-space scale"

Standard map, inverse map, Jacobian, and control segment

The lifted standard map is

```
y_{n+1} = y_n - k/(2*pi) sin(2*pi*x_n),
x_{n+1} = x_n + y_{n+1}.
```

The cylinder version wraps only x. The inverse map is checked by applying the forward map followed by the inverse map. The Jacobian determinant is one, so the map is area-preserving in the lifted variables.

```
zCheck = [0.31; 0.44];
zForward = so_standard_map_lifted(zCheck, cfg);
zBack = so_standard_map_inverse_lifted(zForward, cfg);
J = so_jacobian(zCheck, cfg);
h = 1e-7;
Jfd = [(so_standard_map_lifted(zCheck + [h; 0], cfg) - so_standard_map_lifted(zCheck - [h; 0], cfg)) / (2*h), ...
       (so_standard_map_lifted(zCheck + [0; h], cfg) - so_standard_map_lifted(zCheck - [0; h], cfg)) / (2*h)];
mapValidationTable = table( ...
    norm(zBack - zCheck), ...
    det(J), ...
    norm(J - Jfd), ...
    cfg.controlAmplitude, ...
    2*cfg.controlAmplitude, ...
    'VariableNames', {'forwardInverseError','jacobianDeterminant','jacobianFiniteDifferenceError','uMax','delta'})
```

mapValidationTable = 1×5 table

	forwardInverseError	jacobianDeterminant	jacobianFiniteDifferenceError	uMax	delta
1	7.8505e-17	1.0000	5.8746e-10	0.0030	0.0060

UPO enumeration

Periodic chains are solved using fixed-(p,m) Newton equations in the lifted map,

```
F_lift^p(z) - z - [m; 0] = 0.
```

Multiple distinct chains with the same (p,m) are retained. Lower-period duplicates are rejected, and cyclic/cylindrical deduplication prevents repeated copies of the same chain. Stability is classified from the trace of the monodromy matrix.

```
catalogue = so_enumerate_periodic_orbits(cfg);
upoCatalogue = catalogue.byOmega
```

upoCatalogue = 18×12 table

	id	period	winding	omega	trace	residue	absResidue	residual	averageY	averageYError	classification	samePMCount
1	"chain-012"	5	1	0.2000	-7.4731	2.3683	2.3683	3.2398e-16	0.2000	0	"inverse-hyperbolic"	2
2	"chain-011"	5	1	0.2000	11.8949	-2.4737	2.4737	1.2686e-14	0.2000	3.1641e-15	"direct-hyperbolic"	2
3	"chain-008"	4	1	0.2500	-2.1342	1.0335	1.0335	2.7756e-17	0.2500	1.1102e-16	"inverse-hyperbolic"	2
4	"chain-007"	4	1	0.2500	6.3029	-1.0757	1.0757	4.9651e-16	0.2500	1.1102e-16	"direct-hyperbolic"	2
5	"chain-003"	3	1	0.3333	-0.2285	0.5571	0.5571	1.1124e-14	0.3333	1.7208e-15	"elliptic"	2
6	"chain-004"	3	1	0.3333	4.3090	-0.5772	0.5772	2.2888e-16	0.3333	0	"direct-hyperbolic"	2
7	"chain-013"	5	2	0.4000	-1.4683	0.8671	0.8671	1.8310e-15	0.4000	1.6653e-16	"elliptic"	2
8	"chain-014"	5	2	0.4000	5.6869	-0.9217	0.9217	2.4825e-16	0.4000	1.1102e-16	"direct-hyperbolic"	2
9	"chain-001"	2	1	0.5000	0.4375	0.3906	0.3906	4.2672e-14	0.5000	1.8818e-14	"elliptic"	2
10	"chain-002"	2	1	0.5000	3.6065	-0.4016	0.4016	1.5701e-16	0.5000	0	"direct-hyperbolic"	2
11	"chain-015"	5	3	0.6000	-1.4683	0.8671	0.8671	2.2204e-16	0.6000	1.1102e-16	"elliptic"	2
12	"chain-016"	5	3	0.6000	5.6869	-0.9217	0.9217	5.2369e-16	0.6000	0	"direct-hyperbolic"	2
13	"chain-006"	3	2	0.6667	-0.2285	0.5571	0.5571	5.0197e-14	0.6667	1.0103e-14	"elliptic"	2
14	"chain-005"	3	2	0.6667	4.3090	-0.5772	0.5772	4.9651e-16	0.6667	2.2204e-16	"direct-hyperbolic"	2
15	"chain-009"	4	3	0.7500	-2.1342	1.0335	1.0335	1.0474e-15	0.7500	0	"inverse-hyperbolic"	2
16	"chain-010"	4	3	0.7500	6.3029	-1.0757	1.0757	9.1551e-16	0.7500	1.1102e-16	"direct-hyperbolic"	2
17	"chain-018"	5	4	0.8000	-7.4731	2.3683	2.3683	1.7902e-15	0.8000	1.3323e-15	"inverse-hyperbolic"	2
18	"chain-017"	5	4	0.8000	11.8949	-2.4737	2.4737	6.4541e-14	0.8000	4.4409e-16	"direct-hyperbolic"	2

```
retainedDirectHyperbolic = catalogue.byOmega(catalogue.byOmega.classification == "direct-hyperbolic", :)
```

retainedDirectHyperbolic = 9×12 table

	id	period	winding	omega	trace	residue	absResidue	residual	averageY	averageYError	classification	samePMCount
1	"chain-011"	5	1	0.2000	11.8949	-2.4737	2.4737	1.2686e-14	0.2000	3.1641e-15	"direct-hyperbolic"	2
2	"chain-007"	4	1	0.2500	6.3029	-1.0757	1.0757	4.9651e-16	0.2500	1.1102e-16	"direct-hyperbolic"	2
3	"chain-004"	3	1	0.3333	4.3090	-0.5772	0.5772	2.2888e-16	0.3333	0	"direct-hyperbolic"	2
4	"chain-014"	5	2	0.4000	5.6869	-0.9217	0.9217	2.4825e-16	0.4000	1.1102e-16	"direct-hyperbolic"	2
5	"chain-002"	2	1	0.5000	3.6065	-0.4016	0.4016	1.5701e-16	0.5000	0	"direct-hyperbolic"	2
6	"chain-016"	5	3	0.6000	5.6869	-0.9217	0.9217	5.2369e-16	0.6000	0	"direct-hyperbolic"	2
7	"chain-005"	3	2	0.6667	4.3090	-0.5772	0.5772	4.9651e-16	0.6667	2.2204e-16	"direct-hyperbolic"	2
8	"chain-010"	4	3	0.7500	6.3029	-1.0757	1.0757	9.1551e-16	0.7500	1.1102e-16	"direct-hyperbolic"	2
9	"chain-017"	5	4	0.8000	11.8949	-2.4737	2.4737	6.4541e-14	0.8000	4.4409e-16	"direct-hyperbolic"	2

```
diagnosticFractions = catalogue.diagnosticFractions
```

diagnosticFractions = 7×3 table

	omega	represented	directHyperbolic
1	0.2500	true	true
2	0.3333	true	true
3	0.4000	true	true
4	0.5000	true	true
5	0.6000	true	true
6	0.6667	true	true
7	0.7500	true	true

```
maximumAverageYError = max(catalogue.byOmega.averageYError)
```

```
maximumAverageYError =
1.8818e-14
```

Route construction and source-to-proxy audit

The route is chosen algorithmically from direct-hyperbolic chains whose rotation numbers fall inside the declared source-target y bracket. No chain identifier is hard-coded into the route.

The audit table explicitly checks the corrected nondegeneracy condition: the source starts outside all initial proxy balls, and the entire admissible zero-time control segment also stays outside them.

```
route = so_construct_first_light_route(catalogue, cfg);
routeTable = local_route_table(route)
```

routeTable = 2×6 table

	chainID	omega	period	winding	classification	componentCount
1	"chain-016"	0.6000	5	3	"direct-hyperbolic"	5
2	"chain-005"	0.6667	3	2	"direct-hyperbolic"	3

```
sourceProxyAudit = so_source_proxy_audit(cfg, route);
sourceProxyAudit.byProxy
```

ans = 8×11 table

	routeIndex	chainID	componentID	phasePointIndex	proxyX	proxyY	sourceDistance	controlledSegmentDistance	proxyRadius	directAtZeroWithoutControl	directAtZeroWithAdmissibleControl
1	1	"chain-016"	"chain-016-phase-01"	1	0.6661	0.4958	0.2186	0.2184	0.0300	false	false
2	1	"chain-016"	"chain-016-phase-02"	2	1.3339	0.6677	0.4774	0.4764	0.0300	false	false
3	1	"chain-016"	"chain-016-phase-03"	3	1.8297	0.4958	0.0558	0.0553	0.0300	false	false
4	1	"chain-016"	"chain-016-phase-04"	4	2.5000	0.6703	0.4178	0.4166	0.0300	false	false
5	1	"chain-016"	"chain-016-phase-05"	5	3.1703	0.6703	0.3294	0.3279	0.0300	false	false
6	2	"chain-005"	"chain-005-phase-01"	1	0.7674	0.5348	0.1203	0.1197	0.0300	false	false
7	2	"chain-005"	"chain-005-phase-02"	2	1.5000	0.7326	0.4459	0.4443	0.0300	false	false
8	2	"chain-005"	"chain-005-phase-03"	3	2.2326	0.7326	0.4150	0.4134	0.0300	false	false

```
auditSummary = struct2table(rmfield(sourceProxyAudit, 'byProxy'), 'AsArray', true)
```

auditSummary = 1×7 table

	sourceState	minimumSourceDistance	minimumControlledSegmentDistance	directContainmentPossibleAtN0	proxyRadius	sourceOutsideAllInitialProxies	controlSegmentOutsideAllInitialProxies
1	[0.8844;0.5067]	0.0558	0.0553	false	0.0300	true	true

Forward-backward connection solver

A connection from a source point to a target component is found by a diagonal search in increasing total time. For each split (n_forward, n_backward), the forward image of the vertical control segment and the backward image of the target boundary are adaptively refined and intersected. Intersections are refined inside their original segment brackets. Direct zero-time containment is treated separately and chooses the admissible control with minimum absolute value.

The first provisional connection below is computed only to expose the connection data before the full multistage solve.

```
sourceState = so_rectangle_center(cfg.sourceRectangle);
sourceState(1) = so_wrap_x(sourceState(1));
[firstConnection, firstProfile] = so_resolve_connection(sourceState, route.targetComponents{1}, cfg, so_new_performance_profile());
firstConnectionSummary = local_connection_table(firstConnection)
```

firstConnectionSummary = 1×12 table

	success	nForward	nBackward	totalIterations	controlY	intersectionResidual	directContainment	zeroTimePretest	targetContained	timeMinimumCertified	selectionCertified	targetComponent
1	true	2	0	2	0	0	false	false	true	true	true	"chain-016-phase-05"

```
firstProfile
```

```
firstProfile = 
struct with fields:
                      forwardFamilyBuilds: 1
                     backwardFamilyBuilds: 5
                backwardFamilyCacheReuses: 0
                    indexedCrossingQueries: 29
    exactSegmentTestsAfterSpatialFiltering: 6
                   naiveComparisonsAvoided: 44826
                              jProbesPruned: 0
                             runtimeByStage: []
                        stageRuntimeSeconds: []
                       replayRuntimeSeconds: []
                directContainmentCandidates: 0
                            refinementCalls: 2
                            unresolvedSplits: 0
                       curvePointEvaluations: 0
                          peakCurvePointCount: 450
```

Multistage targeting and adaptive switch rule

The full targeting routine first resolves a provisional connection to the current proxy. It then tests each possible switching index j along that planned path. The selected j minimizes

```
J(j) = j + tau_resolved(w_j -> next target).
```

If j=0 is selected, the current proxy is skipped because the calculated route to the next target is no worse than entering the proxy first. The skip is therefore an objective-based decision, not evidence that the source began inside the proxy.

```
targetingTimer = tic;
result = so_multistage_targeting(cfg, catalogue, route);
result.diagnosticManifolds = so_compute_diagnostic_manifolds(route, cfg);
targetingRuntime = toc(targetingTimer);
fprintf('Corrected deterministic targeting runtime: %.3f seconds\n', targetingRuntime);
```

```
Corrected deterministic targeting runtime: 136.029 seconds
```

```
stageSummary = so_stage_summary_table(result)
```

```
stageSummary = 3×18 table
```

	stage	targetKind	targetID	rotationNumber	connectionSuccess	nForward	nBackward	tauResolved	controlY	targetComponent	targetPhasePointIndex	intersectionResidual	targetContained	selectedSwitchIndex	skipped
1	1	"proxy"	"chain-016"	0.6000	true	2	0	2	0	"chain-016-phase-05"	5	0	true	0	true
2	2	"proxy"	"chain-005"	0.6667	true	8	5	13	0.0021	"chain-005-phase-01"	1	1.3757e-12	true	9	false
3	3	"final"	"final-target"	NaN	true	8	6	14	0.0029	"final-target"	NaN	6.2733e-14	true	NaN	false

```
switchTable = local_switch_table(result.switchEvaluations)
```

```
switchTable = 2×8 table
```

	stage	currentTargetID	nextTargetID	selectedJ	selectedObjective	selectedFinite	prunedProbeCount	probeCount
1	1	"chain-016"	"chain-005"	0	13	true	0	3
2	2	"chain-005"	"final-target"	9	23	true	0	14

```
executedControlTable = result.executedControls
```

```
executedControlTable = 2×21 table
```

	stage	targetID	preControlX	preControlY	postControlX	postControlY	switchStateX	switchStateY	controlY	stageIterations	executedIterations	cumulativeIterations	maxPathwiseReplayError	switchStateReplayError
1	2	"chain-005"	0.8844	0.5067	0.8844	0.5088	6.3626	0.6766	0.0021	9	9	9	0	0
2	3	"final-target"	6.3626	0.6766	6.3626	0.6795	15.2241	0.7200	0.0029	14	14	23	0	0

Independent replay of each accepted stage

Each accepted executed segment stores the pre-control state, the instantaneous control, the post-control state, the planned path, the switch/final state, and pathwise replay errors. Replay starts from the stored pre-control state, applies the stored y-kick once, and advances the lifted standard map one step at a time. It does not call the connection solver and does not reconstruct the control.

```
replayTable = local_replay_table(result)
```

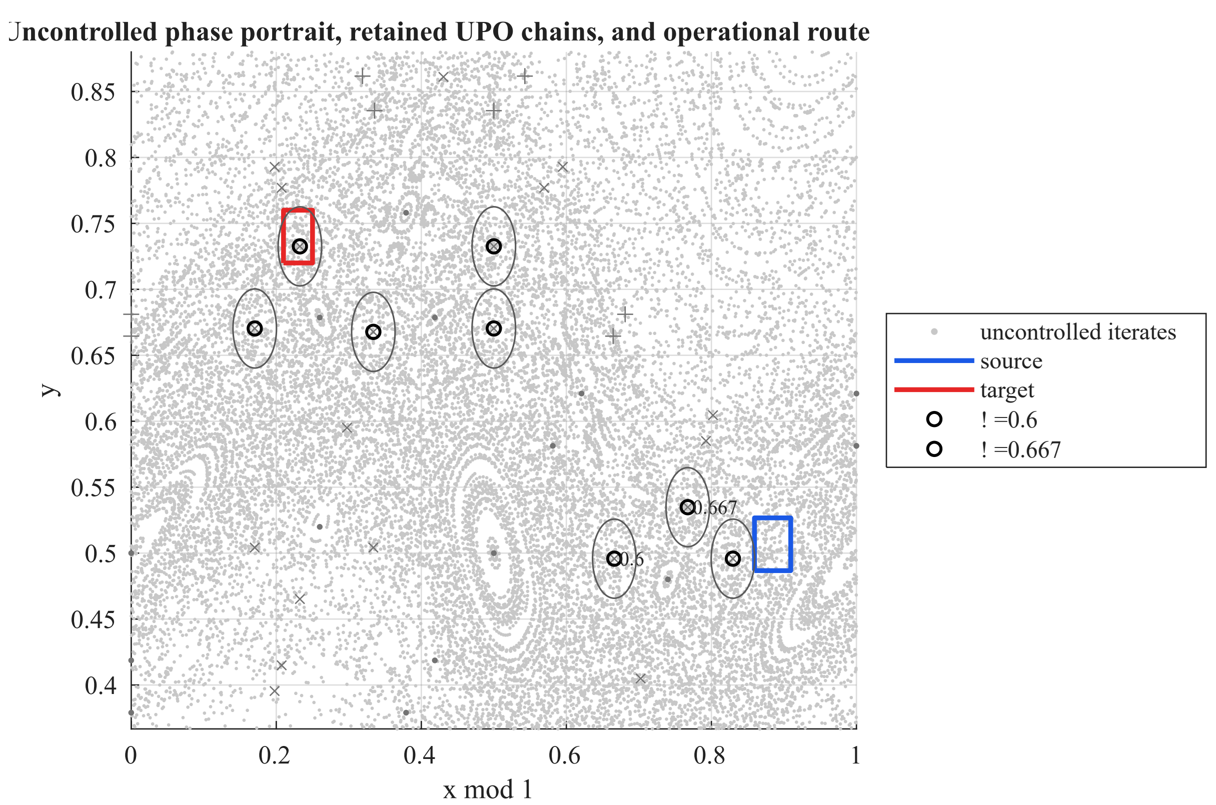
```
replayTable = 2×12 table
```

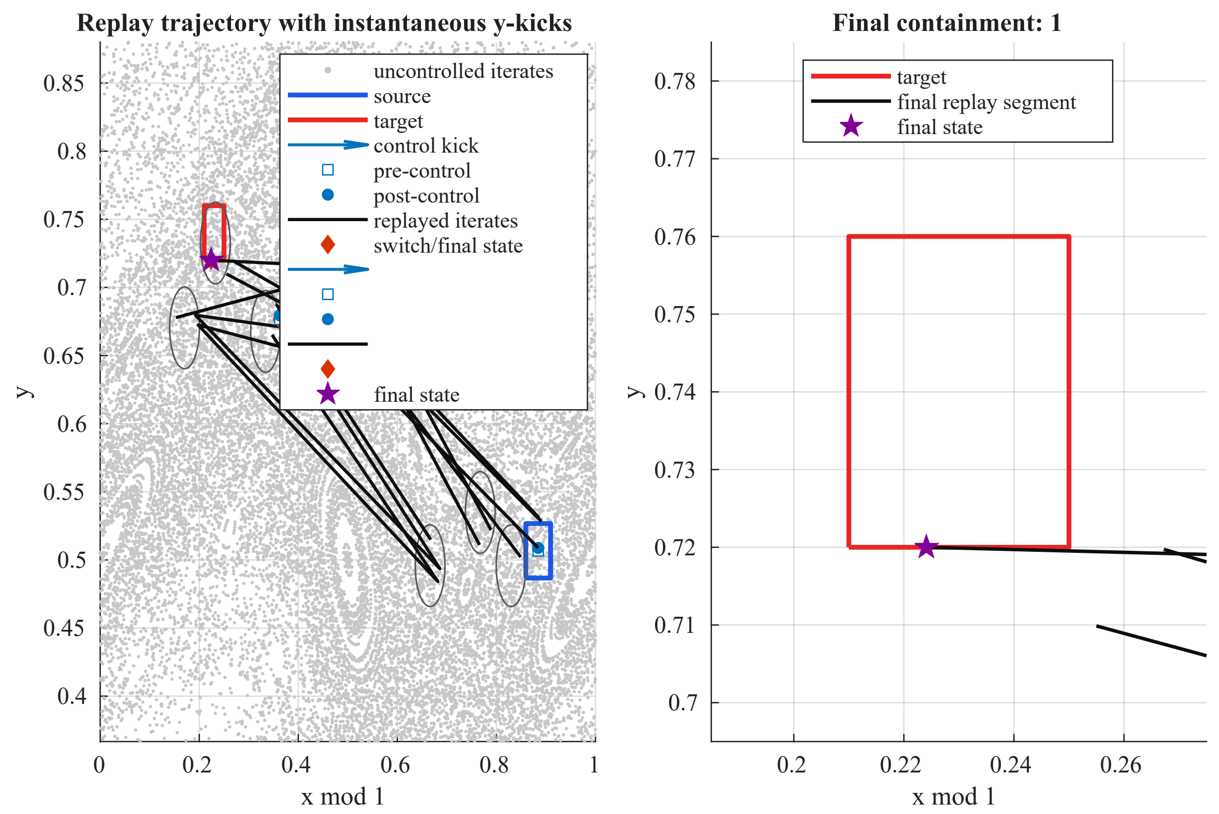
	segment	preControlX	preControlY	controlY	postControlX	postControlY	iterations	maxPathwiseReplayError	switchStateReplayError	finalStateReplayError	targetContained	replayPassed
1	1	0.8844	0.5067	0.0021	0.8844	0.5088	9	0	0	0	false	true
2	2	6.3626	0.6766	0.0029	6.3626	0.6795	14	0	0	0	true	true

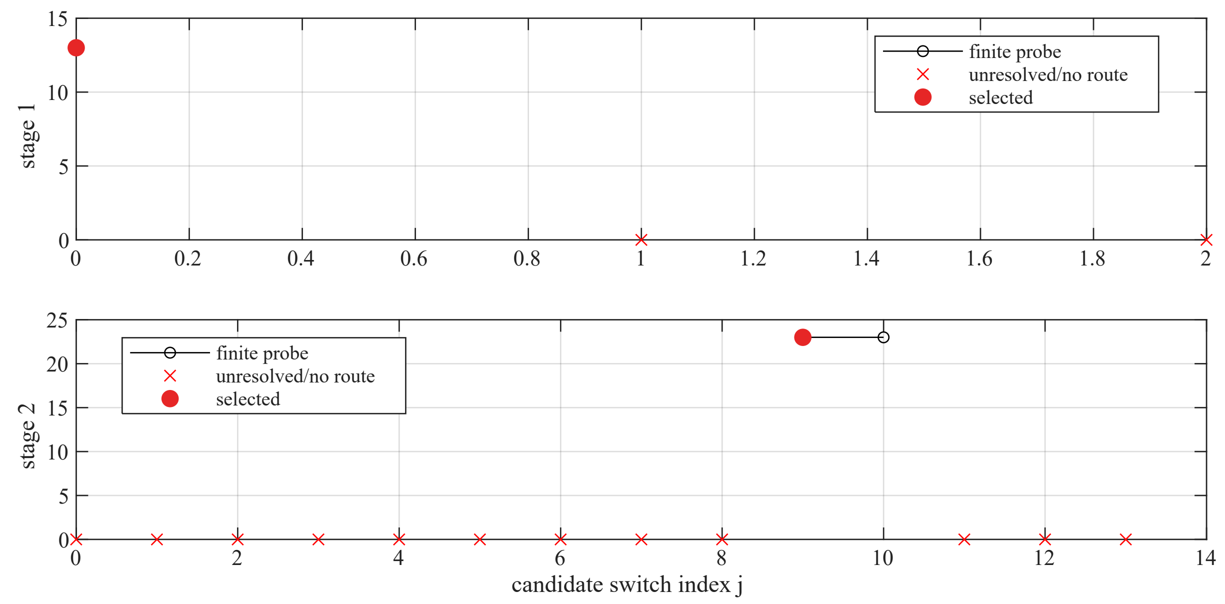
Controlled trajectory figures

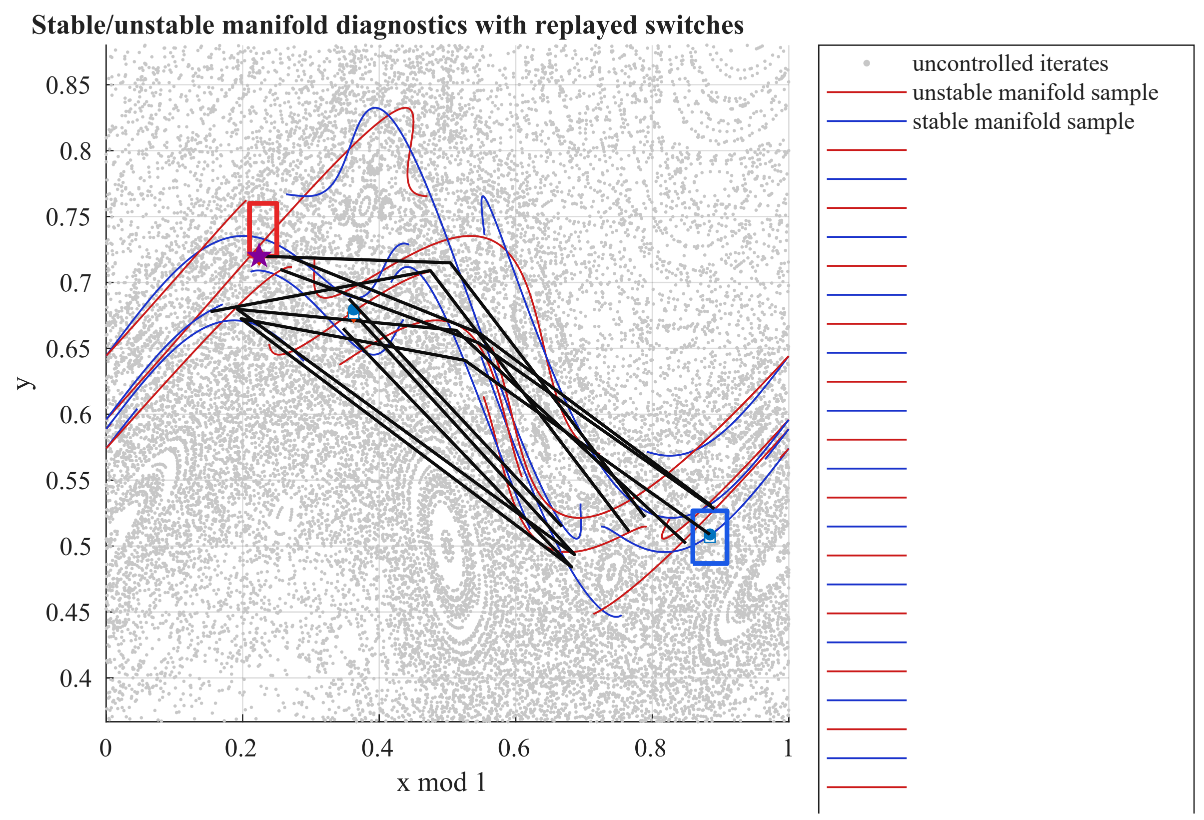
The figures are generated inline. PNG copies are optional, but the important thesis figures are also exported as SVG and EPS in the figure directory. The trajectory figure preserves instantaneous control jumps and inserts NaN breaks across periodic x-boundary jumps.

```
figureHandles = so_plot_results(result);
```










```
exportedFigureInventory = local_export_inventory(cfg.figureDirectory)
```

exportedFigureInventory = 16×2 table

	name	bytes
1	"controlled_trajectory_replay.svg"	19458093
2	"diagnostic_manifolds_switch_points.svg"	19718581
3	"phase_portrait_route.svg"	19600766
4	"switch_objectives.svg"	39082
5	"upo_eigendirections.svg"	77769
6	"controlled_trajectory_replay.eps"	1609102
7	"diagnostic_manifolds_switch_points.eps"	1788815

	name	bytes
8	"phase_portrait_route.eps"	1662665
9	"switch_objectives.eps"	8974
10	"upo_eigendirections.eps"	20130
11	"controlled_trajectory.png"	65457
12	"controlled_trajectory_replay.png"	189989
13	"diagnostic_manifolds_switch_points.png"	247433
14	"phase_portrait_route.png"	220359
15	"switch_objectives.png"	42172
16	"upo_eigendirections.png"	102895

Uncontrolled propagation from the same source point

The uncontrolled baseline is propagated from the same source centre with no controls. Hitting is checked against the final target rectangle over a declared finite observation horizon.

```
uncontrolledHorizon = 5000;
uncontrolledComparison = local_uncontrolled_comparison(sourceState, cfg, uncontrolledHorizon)
```

uncontrolledComparison = 1×5 table

	observationHorizon	reached	hitIteration	lastX	lastY
1	5000	true	1309	375.2440	0.7358

Equation (1) diagnostic

The paper's notation for target size is ambiguous because it refers to a target radius and later to a ball diameter. The code declares the convention in cfg.equationOne.targetSizeConvention. The table also shows the predicted value obtained under the alternative convention. L=1 is labelled as a declared order-one phase-space scale, not a fitted length.

```
equationOneDiagnostics = result.equationOneDiagnostics
```

equationOneDiagnostics = 3×18 table

...

	stage	nForward	nBackward	realisedTau	predictedTau	alternativePredictedTau	delta	epsilon_t	alternativeEpsilon_t	targetSizeConvention	alternativeTargetSizeConvention	L	LConvention	lambda1
1	1	2	0	2	7.4700	7.4700	0.0060	0.0600	0.0300	"diameter"	"radius"	1	"declared order-one phase-space scale"	0.6849
2	2	8	5	13	23.7498	26.5246	0.0060	0.0600	0.0300	"diameter"	"radius"	1	"declared order-one phase-space scale"	0.4097
3	3	8	6	14	22.0230	23.5088	0.0060	0.0400	0.0200	"diameter"	"radius"	1	"declared order-one phase-space scale"	0.3383

Corrected validation suite

The tests include independent negative replay checks: corrupting a stored planned path or corrupting a stored control must fail replay. The test names are distinct and no longer call other tests as aliases.

```
testResults = test_schroer_ott_first_light()
```

name	passed	message
"lifted forward/inverse consistency"	true	""
"cylinder forward/inverse consistency"	true	""
"analytic Jacobian versus finite differences"	true	""
"area preservation"	true	""
"wrapped-x/lifted-x trajectory consistency"	true	""
"y remains unwrapped"	true	""
"adaptive refinement of folded curve"	true	""
"midpoint deviation detects hidden fold"	true	""
"closed-boundary wraparound refinement"	true	""
"indexed intersection versus exhaustive"	true	""
"explicit indexed intersection backend"	true	""
"crossing across x=0"	true	""
"known ordinary polyline crossing"	true	""
"positive-time constructed u0 recovery"	true	""
"zero-time containment minimises absolute control"	true	""
"circular-target containment"	true	""
"rectangle containment"	true	""
"periodic-boundary rectangle containment"	true	""
"diagonal direct time ordering"	true	""
"unresolved curve budget is recorded"	true	""
"unresolved numerical status does not throw"	true	""
"incremental family versus independent recomputation"	true	""
"shared backward cache versus independent recomputation"	true	""
"union target minimises over phase components"	true	""
"phase-point tie-breaking"	true	""
"diagonal search returns finite constructed result"	true	""
"fixed-(p,m) periodic residuals"	true	""
"retained-chain average-y consistency"	true	""
"multiple chains retained for same (p,m)"	true	""
"direct-hyperbolic omega=1/4 recovered"	true	""
"cyclic/cylinder deduplication"	true	""
"inverse-hyperbolic chains excluded from route"	true	""
"elliptic chains excluded from route"	true	""
"near-parabolic classification"	true	""
"declared route bracket policy"	true	""

```
    "source is outside all initial proxies"                  true       ""
    "executed control bounds"                                true       ""
    "control bookkeeping columns are distinct"               true       ""
    "independent replay validates accepted stages"           true       ""
    "replay detects corrupted planned path"                  true       ""
    "replay detects corrupted control"                       true       ""
    "j=0 switch mechanism is objective-based"                true       ""
    "at least one positive switching segment executes"       true       ""
    "exact pruning rule"                                     true       ""
    "executed-time accounting"                               true       ""
    "final target independent containment"                   true       ""
testResults = 46×3 table
```

	name	passed	message
1	"lifted forward/inverse consistency"	true	""
2	"cylinder forward/inverse consistency"	true	""
3	"analytic Jacobian versus finite differences"	true	""
4	"area preservation"	true	""
5	"wrapped-x/lifted-x trajectory consistency"	true	""
6	"y remains unwrapped"	true	""
7	"adaptive refinement of folded curve"	true	""
8	"midpoint deviation detects hidden fold"	true	""
9	"closed-boundary wraparound refinement"	true	""
10	"indexed intersection versus exhaustive"	true	""
11	"explicit indexed intersection backend"	true	""
12	"crossing across x=0"	true	""
13	"known ordinary polyline crossing"	true	""
14	"positive-time constructed u0 recovery"	true	""
15	"zero-time containment minimises absolute control"	true	""
16	"circular-target containment"	true	""
17	"rectangle containment"	true	""
18	"periodic-boundary rectangle containment"	true	""
19	"diagonal direct time ordering"	true	""
20	"unresolved curve budget is recorded"	true	""
21	"unresolved numerical status does not throw"	true	""
22	"incremental family versus independent recomputation"	true	""
23	"shared backward cache versus independent recomputation"	true	""
24	"union target minimises over phase components"	true	""
25	"phase-point tie-breaking"	true	""
26	"diagonal search returns finite constructed result"	true	""
27	"fixed-(p,m) periodic residuals"	true	""
28	"retained-chain average-y consistency"	true	""
29	"multiple chains retained for same (p,m)"	true	""
30	"direct-hyperbolic omega=1/4 recovered"	true	""
31	"cyclic/cylinder deduplication"	true	""
32	"inverse-hyperbolic chains excluded from route"	true	""
33	"elliptic chains excluded from route"	true	""
34	"near-parabolic classification"	true	""
35	"declared route bracket policy"	true	""
36	"source is outside all initial proxies"	true	""
37	"executed control bounds"	true	""
38	"control bookkeeping columns are distinct"	true	""
39	"independent replay validates accepted stages"	true	""
40	"replay detects corrupted planned path"	true	""
41	"replay detects corrupted control"	true	""
42	"j=0 switch mechanism is objective-based"	true	""
43	"at least one positive switching segment executes"	true	""
44	"exact pruning rule"	true	""
45	"executed-time accounting"	true	""
46	"final target independent containment"	true	""

```
testSummary = groupsummary(testResults, "passed")
```

testSummary = 1×2 table

	passed	GroupCount
1	true	46

Three-level numerical-resolution study

This compact resolution study is project-specific. It varies only curve and intersection-resolution settings for the corrected deterministic first-light case. It does not change k, source, target, control bound, route policy, or iteration horizons.

```
resolutionStudy = so_run_resolution_study(true);
resolutionStudy.comparison
```

ans = 3×21 table

...

	level	initialControlSamples	initialBoundarySamples	maxGapFraction	midpointToleranceFraction	maxSubdivisionDepth	maxPoints	success	failureCategory	selectedRoute	controls	splits	switchingIndices
1	"reduced"	13	48	0.1400	0.0700	15	50000	true	""	'[0.6 0.666667]'	2	"[2 0]; [8 5]; [8 6]"	'[0 9 NaN]'
2	"baseline"	17	64	0.1200	0.0600	15	100000	true	""	'[0.6 0.666667]'	2	"[2 0]; [8 5]; [8 6]"	'[0 9 NaN]'
3	"increased"	21	80	0.1000	0.0500	15	120000	true	""	'[0.6 0.666667]'	2	"[2 0]; [7 6]; [8 6]"	'[0 9 NaN]'

Estimated paper Figure-3 comparison case

The source and target rectangles for the paper comparison are estimated from the rendered Figure 3 axes because the paper does not tabulate their exact coordinates. The seven route rotations are selected from the direct-hyperbolic chains matching the full-width resonances visible in the figure: 1/4, 1/3, 2/5, 1/2, 3/5, 2/3, and 3/4.

The published 125-132 iteration range is reported descriptively. It is not used as an acceptance criterion.

```
[paperCfg, paperMetadata] = so_paper_comparison_config();
paperMetadata
```

```
paperMetadata = 
struct with fields:
                      source: [1×1 struct]
                      target: [1×1 struct]
          digitisationMethod: "visual reading of rendered Schroer-Ott Figure 3 page against axes"
       coordinateUncertainty: 0.0150
     expectedRotationNumbers: [0.2500 0.3333 0.4000 0.5000 0.6000 0.6667 0.7500]
    publishedControlledRange: [125 132]
   publishedExampleIterations: 125
         maxControlAmplitude: 0.0030
```

```
paperCase = so_run_paper_comparison_case(true);
paperComparisonSummary = local_paper_case_table(paperCase)
```

paperComparisonSummary = 1×5 table

	classification	targetReached	totalIterations	numberOfControls	targetContained
1	"failed numerical reproduction"	false	0	0	false

Fifty-source paper-comparison ensemble

The following command runs exactly 50 uniformly sampled source points in the adopted paper-comparison source rectangle. It stores all successes, failures, unresolved cases, exact sampled states, replay errors, and comparison counts relative to the published 125-132 range.

This can be computationally expensive because every trial runs the full forward-backward route independently. It is intentionally not replaced by three hand-picked points.

```
paperEnsembleFile = fullfile(paperCfg.outputDirectory, 'paper_50_source_ensemble.mat');
runPaperEnsembleNow = exist('RUN_FULL_PAPER_ENSEMBLE', 'var') && RUN_FULL_PAPER_ENSEMBLE;
if runPaperEnsembleNow || ~isfile(paperEnsembleFile)
    paperEnsemble = so_run_paper_ensemble(true);
else
    loadedEnsemble = load(paperEnsembleFile, 'ensemble');
    paperEnsemble = loadedEnsemble.ensemble;
end
paperEnsemble.summary
```

ans = 1×13 table

	attempts	successes	successFraction	minimum	maximum	mean	median	standardDeviation	interquartileRange	finiteIterationCounts	numberBelow125	numberWithin125To132	numberAbove132
1	50	0	0	NaN	NaN	NaN	NaN	NaN	NaN	""	0	0	0

```
paperEnsemble.trials
```

ans = 50×18 table

...

	trial	sourceX	sourceY	success	failureCategory	selectedRoute	numberOfControls	totalControlledIterations	maxAbsControl	totalAbsControl	maxIntersectionResidual	maxPathwiseReplayError	finalX
1	1	0.4762	0.0335	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
2	2	0.4809	0.0409	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
3	3	0.4974	0.0425	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
4	4	0.5270	0.0374	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
5	5	0.5242	0.0115	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN

	trial	sourceX	sourceY	success	failureCategory	selectedRoute	numberOfControls	totalControlledIterations	maxAbsControl	totalAbsControl	maxIntersectionResidual	maxPathwiseReplayError	finalX
6	6	0.5278	0.0021	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
7	7	0.4838	0.0156	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
8	8	0.4804	0.0102	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
9	9	0.5511	0.0256	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
10	10	0.5115	0.0309	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
11	11	0.4617	0.0140	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
12	12	0.5283	0.0309	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
13	13	0.5060	0.0296	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
14	14	0.4778	0.0346	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
15	15	0.4891	0.0187	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
16	16	0.5062	0.0435	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
17	17	0.5511	0.0284	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
18	18	0.5164	0.0078	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	1	1	NaN	NaN	NaN	NaN	NaN
19	19	0.5302	0.0254	false	"provisional_connection_failed"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
20	20	0.5450	0.0330	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
21	21	0.4897	0.0383	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
22	22	0.5205	0.0442	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
23	23	0.4947	0.0184	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
24	24	0.4677	0.0038	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
25	25	0.5008	0.0217	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
26	26	0.4686	0.0281	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
27	27	0.4748	0.0078	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
28	28	0.4861	0.0063	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
29	29	0.5035	0.0293	false	"all_switch_probes_failed"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
30	30	0.4997	0.0262	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
31	31	0.5283	0.0359	false	"all_switch_probes_failed"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
32	32	0.5008	0.0086	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
33	33	0.5053	0.0299	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	1	1	NaN	NaN	NaN	NaN	NaN
34	34	0.4598	0.0418	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
35	35	0.5095	0.0393	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
36	36	0.5408	0.0087	false	"provisional_connection_failed"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
37	37	0.4992	0.0135	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
38	38	0.5146	0.0113	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	1	1	NaN	NaN	NaN	NaN	NaN
39	39	0.5038	0.0064	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
40	40	0.4611	0.0227	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
41	41	0.4692	0.0201	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
42	42	0.4992	0.0359	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
43	43	0.5188	0.0225	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
44	44	0.5004	0.0187	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
45	45	0.5530	0.0388	false	"provisional_connection_failed"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
46	46	0.5047	0.0208	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
47	47	0.5538	0.0068	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
48	48	0.5494	0.0173	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
49	49	0.4863	0.0209	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN
50	50	0.5077	0.0138	false	"runtime_limit_exceeded"	'[0.25 0.333333 0.4 0.5 0.6 0.666667 0.75]'	0	0	NaN	NaN	NaN	NaN	NaN

One mild-noise demonstration

The deterministic map functions are not changed for noise. Instead, a separate replay wrapper applies the already-computed deterministic control schedule and adds one stored Gaussian y-kick sequence with sigma = 0.01*(2*u_max) = 6e-5. No retargeting is performed.

```
noiseDemo = so_run_mild_noise_demo(result, true);
noiseSummary = local_noise_table(noiseDemo)
```

noiseSummary = 1×11 table

	sigma	delta	seed	noisySuccess	totalIterations	maximumControl	storedNoiseReplayError	finalX	finalY	finalContainment	endpointDisplacement
1	6.0000e-05	0.0060	314159	false	23	0.0029	0	15.2706	0.7447	false	0.0527

Final validation summary

The table below collects the main deterministic checks. Paper-comparison and 50-source ensemble outcomes must be interpreted separately because the paper geometry is estimated from a figure, not tabulated.

```
finalValidationSummary = local_final_validation_table(result, sourceProxyAudit, testResults, resolutionStudy, paperCase, paperEnsemble, noiseDemo)
```

finalValidationSummary = 13×2 table

	quantity	value
1	"deterministicTargetReached"	"true"
2	"sourceOutsideInitialProxies"	"true"
3	"controlSegmentOutsideInitialProxies"	"true"
4	"anyPositiveSwitch"	"true"
5	"allControlsWithinBound"	"true"
6	"maxReplayError"	"0"
7	"finalContainment"	"true"
8	"testSuitePassed"	"true"
9	"resolutionLevelsSucceeded"	"true"
10	"paperCaseClassification"	"failed numerical reproduction"
11	"paperEnsembleAttempts"	"50"
12	"paperEnsembleSuccesses"	"0"
13	"noiseStoredReplayError"	"0"

Local helper functions

```
function inventory = local_file_inventory(projectFolder)
files = dir(fullfile(projectFolder, '*'));
isRegular = ~[files.isdir].';
files = files(isRegular);
inventory = table(string({files.name}).', string({files.folder}).', [files.bytes].', ...
    datetime([files.datenum].', 'ConvertFrom', 'datenum'), ...
    'VariableNames', {'name','folder','bytes','lastModified'});
inventory = sortrows(inventory, 'name');
end

function tbl = local_configuration_table(cfg)
names = ["k";"controlAmplitude";"sourceX";"sourceY";"targetX";"targetY"; ...
    "maxForwardIterations";"maxBackwardIterations";"maxTotalTransferTime"; ...
    "proxyTargetRadius";"routeBracketMargin";"routeLowerBoundary";"routeUpperBoundary"; ...
    "intersectionBackend";"equationOneTargetSizeConvention";"equationOneLConvention"];
values = [
    string(cfg.k)
    string(cfg.controlAmplitude)
    sprintf('[%.6f, %.6f]', cfg.sourceRectangle.xMin, cfg.sourceRectangle.xMax)
    sprintf('[%.6f, %.6f]', cfg.sourceRectangle.yMin, cfg.sourceRectangle.yMax)
    sprintf('[%.6f, %.6f]', cfg.targetRectangle.xMin, cfg.targetRectangle.xMax)
    sprintf('[%.6f, %.6f]', cfg.targetRectangle.yMin, cfg.targetRectangle.yMax)
    string(cfg.maxForwardIterations)
    string(cfg.maxBackwardIterations)
    string(cfg.maxTotalTransferTime)
    string(cfg.proxyTargetRadius)
    string(cfg.routeBracket.margin)
    string(cfg.routeBracket.lowerBoundary)
    string(cfg.routeBracket.upperBoundary)
    string(cfg.intersectionBackend)
    string(cfg.equationOne.targetSizeConvention)
    string(cfg.equationOne.LConvention)
    ];
tbl = table(names, values, 'VariableNames', {'quantity','value'});
end

function tbl = local_route_table(route)
chainID = string({route.chains.id}).';
omega = [route.chains.omega].';
period = [route.chains.period].';
winding = [route.chains.winding].';
classification = string({route.chains.classification}).';
componentCount = cellfun(@numel, route.targetComponents).';
tbl = table(chainID, omega, period, winding, classification, componentCount);
```

```
end

function tbl = local_connection_table(conn)
tbl = table(conn.success, conn.nForward, conn.nBackward, conn.totalIterations, ...
    conn.control, conn.intersectionResidual, conn.directContainment, ...
    conn.zeroTimePretest, conn.targetContained, conn.timeMinimumCertified, ...
    conn.selectionCertified, string(conn.targetComponent), ...
    'VariableNames', {'success','nForward','nBackward','totalIterations','controlY', ...
    'intersectionResidual','directContainment','zeroTimePretest','targetContained', ...
    'timeMinimumCertified','selectionCertified','targetComponent'});
end

function tbl = local_switch_table(switchEvaluations)
rows = {};
for i = 1:numel(switchEvaluations)
    selected = switchEvaluations(i).selected;
    rows(end + 1, :) = {switchEvaluations(i).stage, switchEvaluations(i).currentTargetID, ...
        switchEvaluations(i).nextTargetID, selected.j, selected.objectiveJ, ...
        selected.finite, switchEvaluations(i).pruned, numel(switchEvaluations(i).probes)}; %#ok<AGROW>
end
tbl = cell2table(rows, 'VariableNames', {'stage','currentTargetID','nextTargetID', ...
    'selectedJ','selectedObjective','selectedFinite','prunedProbeCount','probeCount'});
end

function tbl = local_replay_table(result)
rows = {};
for i = 1:numel(result.executionSegments)
    seg = result.executionSegments(i);
    rows(end + 1, :) = {i, seg.preControlState(1), seg.preControlState(2), ...
        seg.control, seg.postControlState(1), seg.postControlState(2), ...
        seg.iterations, seg.maxPathwiseReplayError, seg.switchStateReplayError, ...
        seg.finalStateReplayError, seg.targetContained, seg.replayPassed}; %#ok<AGROW>
end
tbl = cell2table(rows, 'VariableNames', {'segment','preControlX','preControlY', ...
    'controlY','postControlX','postControlY','iterations','maxPathwiseReplayError', ...
    'switchStateReplayError','finalStateReplayError','targetContained','replayPassed'});
end

function exported = local_export_inventory(figureDirectory)
files = [dir(fullfile(figureDirectory, '*.svg')); dir(fullfile(figureDirectory, '*.eps')); ...
    dir(fullfile(figureDirectory, '*.png'))];
if isempty(files)
    exported = table();
else
    exported = table(string({files.name}).', [files.bytes].', ...
        'VariableNames', {'name','bytes'});
end
end

function tbl = local_uncontrolled_comparison(z0, cfg, horizon)
target = so_make_rectangle_component(cfg.targetRectangle, cfg, 'final-target');
z = z0;
hitIteration = NaN;
for n = 1:horizon
    z = so_standard_map_lifted(z, cfg);
    if target.contains(z, cfg.containmentTolerance)
        hitIteration = n;
        break;
    end
end
tbl = table(horizon, ~isnan(hitIteration), hitIteration, z(1), z(2), ...
    'VariableNames', {'observationHorizon','reached','hitIteration','lastX','lastY'});
end

function tbl = local_paper_case_table(paperCase)
if isempty(paperCase.result)
    tbl = table(string(paperCase.classification), false, Inf, 0, false, ...
        'VariableNames', {'classification','targetReached','totalIterations','numberOfControls','targetContained'});
else
    tbl = table(string(paperCase.classification), paperCase.result.targetReached, ...
        paperCase.result.totalExecutedIterations, paperCase.result.numberOfControls, ...
        paperCase.result.targetContained, ...
        'VariableNames', {'classification','targetReached','totalIterations','numberOfControls','targetContained'});
end
end

function tbl = local_noise_table(noiseDemo)
tbl = table(noiseDemo.sigma, noiseDemo.delta, noiseDemo.seed, noiseDemo.noisySuccess, ...
```

```
    noiseDemo.totalIterations, noiseDemo.maximumControl, noiseDemo.storedNoiseReplayError, ...
    noiseDemo.finalState(1), noiseDemo.finalState(2), noiseDemo.finalContainment, ...
    noiseDemo.endpointDisplacement, ...
    'VariableNames', {'sigma','delta','seed','noisySuccess','totalIterations', ...
    'maximumControl','storedNoiseReplayError','finalX','finalY','finalContainment', ...
    'endpointDisplacement'});
end

function tbl = local_final_validation_table(result, audit, testResults, resolutionStudy, paperCase, paperEnsemble, noiseDemo)
quantity = [
    "deterministicTargetReached"
    "sourceOutsideInitialProxies"
    "controlSegmentOutsideInitialProxies"
    "anyPositiveSwitch"
    "allControlsWithinBound"
    "maxReplayError"
    "finalContainment"
    "testSuitePassed"
    "resolutionLevelsSucceeded"
    "paperCaseClassification"
    "paperEnsembleAttempts"
    "paperEnsembleSuccesses"
    "noiseStoredReplayError"
    ];
value = [
    string(result.targetReached)
    string(audit.sourceOutsideAllInitialProxies)
    string(audit.controlSegmentOutsideAllInitialProxies)
    string(any(result.executedControls.stageIterations > 0))
    string(all(result.executedControls.controlWithinBound))
    string(max(result.executedControls.maxPathwiseReplayError))
    string(result.targetContained)
    string(all(testResults.passed))
    string(all(resolutionStudy.comparison.success))
    string(paperCase.classification)
    string(paperEnsemble.summary.attempts)
    string(paperEnsemble.summary.successes)
    string(noiseDemo.storedNoiseReplayError)
    ];
tbl = table(quantity, value);
end
```